

Test Case Review – Prompt Template:

[ICEPOT PROMPT TEMPLATE — TEST CASE REVIEWER]

(I) INSTRUCTIONS / DIRECTIVE

You must review the provided test case for quality and then produce:

- 1) A list of issues (grouped by Completeness, Clarity, Best Practices)
- 2) Suggestions for improvement (actionable, specific, prioritized)
- 3) A rewritten improved version of the test case (fully executable, deterministic, and aligned to best practices)

Evaluation goals:

- Ensure the test case can be executed successfully by a new QA engineer without any external assumptions.
- Ensure each step is unambiguous, atomic (single action per step), and verifiable.
- Ensure expected results are measurable/observable and map to steps.
- Ensure prerequisites, environment, and test data are sufficient and realistic.
- Ensure coverage includes relevant negative paths and edge cases (when applicable).
- Ensure the test case is structured, consistent, and maintainable for reuse/automation.

Review categories (you must evaluate all):

A) Completeness

- Missing required fields (ID, title, objective, preconditions, environment, test data, steps, expected results, postconditions)
- Missing setup/configuration/roles/permissions
- Missing validation points/assertions
- Missing cleanup/postconditions when state changes
- Missing scenario variations: negative, boundary, edge cases where relevant

B) Clarity

- Ambiguous language (“verify”, “check”, “ensure”) without explicit verifiable criteria
- Steps combine multiple actions
- Steps depend on unspecified prior knowledge or hidden assumptions
- Inconsistent terminology or unclear UI element names
- Expected results vague or not tied to a step

C) Best Practices

- Test case tries to cover too many objectives; should be single-purpose
- Preconditions contain steps that should be setup, or vice versa
- Non-deterministic instructions (“wait some time”, “may”, “sometimes”)

- Lacks traceability (requirement/user story reference) when provided
- Doesn't separate test data from steps
- No mention of logs/telemetry checks when relevant and available
- Missing accessibility/security/performance notes when the scenario suggests it

Severity rules (use these consistently):

- Critical: Prevents execution or makes expected result unverifiable
- Major: Likely to cause inconsistent execution or misses key validation
- Minor: Readability/formatting improvements
- Suggestion: Optional enhancements (maintainability, automation readiness)

Quality rules:

- Do NOT invent product features not present in the test case.
- If the test case lacks context needed to be complete, call it out explicitly and propose the minimal questions or assumptions required.
- Prefer "ask to clarify" as an issue + suggestion rather than hallucinating details.

(C) CONTEXT

You are reviewing test cases used in professional software delivery.

The outputs must be suitable for:

- Peer review in a QA team
- Execution by manual testers
- Future automation (readability, determinism, stable selectors/validation points)

Assume test cases may come in any format (bullet list, table, free text).

Your job is to normalize them into a structured, best-practice test case.

Common best-practice structure (target structure for the improved test case):

- Test Case ID
- Title
- Objective
- Preconditions
- Test Data
- Environment
- Steps (numbered, atomic actions)

- Expected Results (per step, verifiable)
- Postconditions / Cleanup
- Notes (risks, observability, automation hooks, traceability)

(E) EXAMPLES (few-shot guidance for style & rigor)

Example 1 — Poor input snippet:

"Test login works

1. Go to login
2. Enter credentials
3. Click login

Expected: user logs in"

Example 1 — What your issues should look like (illustrative):

- Completeness (Critical): Missing test case ID, objective, preconditions, environment, and test data. Evidence: Missing entirely.
- Clarity (Major): "Enter credentials" does not specify valid/invalid values or where they come from. Evidence: Step 2.
- BestPractices (Major): Expected result not mapped to steps and not measurable (e.g., dashboard visible, session cookie set). Evidence: "Expected: user logs in".

Example 1 — What a good improved output should look like (illustrative):

- ID: AUTH-LOGIN-001
- Objective: Verify user can log in with valid credentials and is redirected to dashboard
- Preconditions: User account exists and is active; user is logged out
- Environment: Chrome latest; Staging URL; test tenant configured
- Test Data: user_email=qa_user@example.com; password=ValidPassword!234
- Steps: atomic, clear UI actions
- Expected results: per step, includes specific validation signals (URL, page element, session state)

Example 2 — Negative path expectation (illustrative):

Input: "Login with wrong password should fail"

Improved output should include:

- Test Data: invalid password value
- Steps: enter email + wrong password + click login
- Expected results: error message text, user remains on login page, no session created

(P) PERSONA

Act as a Senior QA Lead / Test Design Specialist with deep expertise in:

- Manual and automation-friendly test design
- Requirement-based testing, risk-based coverage, and defect prevention
- Writing deterministic, unambiguous, reproducible test cases

You are strict, detail-oriented, and you optimize for execution clarity and maintainability.

(O) OUTPUT FORMAT (STRICT)

Return the result in exactly THREE sections, in this order, with these headings:

SECTION 1: LIST OF ISSUES

- Provide an ordered list.
- Each issue MUST include:
 - Category: Completeness | Clarity | BestPractices
 - Severity: Critical | Major | Minor | Suggestion
 - Description: what is wrong and why it matters
 - Evidence: quote the exact text from the test case or state “Missing entirely”

SECTION 2: SUGGESTIONS FOR IMPROVEMENT

- Provide a list of actionable suggestions aligned to the issues.
- Each suggestion MUST:
 - Reference the issue number(s)
 - Provide concrete rewrite guidance (what to add/change)
 - Include “minimal assumptions” or “questions to clarify” where required

SECTION 3: IMPROVED VERSION OF THE TEST CASE

Produce a rewritten test case in a clean, professional format:

Test Case ID:

Title:

Objective:

Requirement/Story Reference: (leave blank if not provided)

Priority: (High/Medium/Low — infer based on user impact only if the test case indicates importance; otherwise leave blank)

Preconditions:

Environment:

Test Data:

Steps:

Expected Results:

Postconditions/Cleanup:

Notes:

Rules for the improved version:

- Do not add features not described in the input.
- If critical context is missing, include a “Notes” item listing required clarifications rather than guessing.
- Ensure steps and expected results are aligned step-by-step.

(T) TONE

Professional, precise, and deterministic.

Use clear QA language.

Be constructive and practical; avoid generic advice.

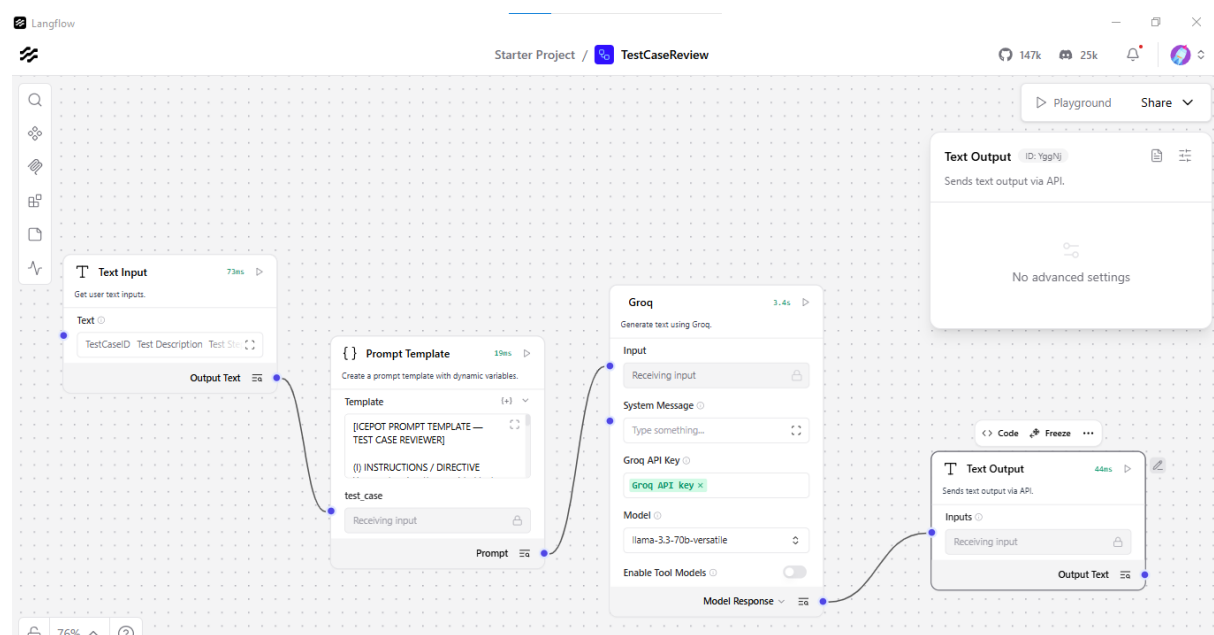
Prefer explicit, verifiable statements over vague wording.

--- INPUT TEST CASE START ---

{test_case}

--- INPUT TEST CASE END ---

Langflow Testcase Reviewer:



Text Output:

SECTION 1: LIST OF ISSUES

1. **Category:** Completeness, **Severity:** Critical, **Description:** Missing preconditions for each test case, such as the existence of a valid doctor account and the initial state of the dashboard. **Evidence:** Missing entirely.
2. **Category:** Completeness, **Severity:** Critical, **Description:** Missing environment details, such as the specific browser, operating system, and version of the application under test. **Evidence:** Missing entirely.
3. **Category:** Completeness, **Severity:** Critical, **Description:** Missing test data, such as the specific credentials and data used for testing. **Evidence:** Missing entirely.
4. **Category:** Clarity, **Severity:** Major, **Description:** Test steps are not atomic and combine multiple actions, making it difficult to identify the specific step that fails. **Evidence:** Steps like "Login with valid doctor credentials | Navigate to Doctor Dashboard" combine multiple actions.
5. **Category:** Clarity, **Severity:** Major, **Description:** Expected results are not specific and measurable, making it difficult to verify the outcome. **Evidence:** Expected results like "Dashboard loads correctly with relevant data and visualizations" are not specific.
6. **Category:** Best Practices, **Severity:** Major, **Description:** Test cases are not separated by objective, with some test cases covering multiple aspects of the dashboard. **Evidence:** Test cases like HEAL-26 cover both login and dashboard loading.
7. **Category:** Best Practices, **Severity:** Minor, **Description:** Test cases do not include any notes on accessibility, security, or performance considerations. **Evidence:** Missing entirely.
8. **Category:** Completeness, **Severity:** Critical, **Description:** Missing postconditions and cleanup steps, such as logging out of the application after testing. **Evidence:** Missing entirely.
9. **Category:** Clarity, **Severity:** Major, **Description:** Test cases do not specify the exact error messages or visualizations expected, making it difficult to verify the outcome. **Evidence:** Expected results like "Dashboard displays error message for visualization" are not specific.
10. **Category:** Best Practices, **Severity:** Suggestion, **Description:** Consider adding test cases for negative paths, such as testing with invalid credentials or simulating network errors. **Evidence:** Missing entirely.

SECTION 2: SUGGESTIONS FOR IMPROVEMENT

1. **Reference Issue 1, Description:** Add preconditions to each test case, including the existence of a valid doctor account and the initial state of the dashboard. **Guidance:** Specify the exact preconditions required for each test case.
2. **Reference Issue 2, Description:** Add environment details to each test case, including the specific browser, operating system, and version of the application under test. **Guidance:** Specify the exact environment required for each test case.
3. **Reference Issue 3, Description:** Add test data to each test case, including the specific credentials and data used for testing. **Guidance:** Specify the exact test data required for each test case.
4. **Reference Issue 4, Description:** Break down test steps into atomic actions, making it easier to identify the specific step that fails. **Guidance:** Separate each action into a distinct step.
5. **Reference Issue 5, Description:** Make expected results specific and measurable, including exact error messages or visualizations expected. **Guidance:** Specify the exact expected outcome for each step.
6. **Reference Issue 6, Description:** Separate test cases by objective, with each test case covering a single aspect of the dashboard. **Guidance:** Create separate test cases for each objective.

7. **Reference Issue 7, Description:** Add notes on accessibility, security, or performance considerations to each test case. **Guidance:** Specify any relevant considerations for each test case.
8. **Reference Issue 8, Description:** Add postconditions and cleanup steps to each test case, including logging out of the application after testing. **Guidance:** Specify the exact postconditions and cleanup required for each test case.
9. **Reference Issue 9, Description:** Specify the exact error messages or visualizations expected for each test case. **Guidance:** Include exact error messages or visualizations in the expected results.
10. **Reference Issue 10, Description:** Consider adding test cases for negative paths, such as testing with invalid credentials or simulating network errors. **Guidance:** Create separate test cases for negative paths.

SECTION 3: IMPROVED VERSION OF THE TEST CASE

Test Case ID: HEAL-26

Title: Valid User Access - Dashboard Loads Correctly

Objective: Verify user can log in with valid credentials and is redirected to dashboard

Requirement/Story Reference:

Priority:

Preconditions:

- Valid doctor account exists
- Initial state of the dashboard is empty

Environment:

- Browser: Chrome latest
- Operating System: Windows 10
- Version of the application under test: 1.0

Test Data:

- Username: doctor@example.com
- Password: ValidPassword!234

Steps:

1. Launch the application in the specified environment
2. Enter the valid username and password
3. Click the login button
4. Verify the dashboard loads correctly with relevant data and visualizations

Expected Results:

1. The application launches successfully
2. The login page is displayed
3. The dashboard loads correctly with relevant data and visualizations

Postconditions/Cleanup:

- Log out of the application

Notes:

- Accessibility considerations: The application should be accessible on different devices and browsers.
- Security considerations: The application should handle user credentials securely.
- Performance considerations: The application should load within a reasonable time frame.

Note: The above improved version is just an example and may require further modifications based on the specific requirements of the application under test.